

「2026년도 2학기 한국성서대학교 AI융합학부」

AIoT프로그래밍

: 능동부저, 수동부저, 텍스트 스위치, 가변저항

5주차



목차

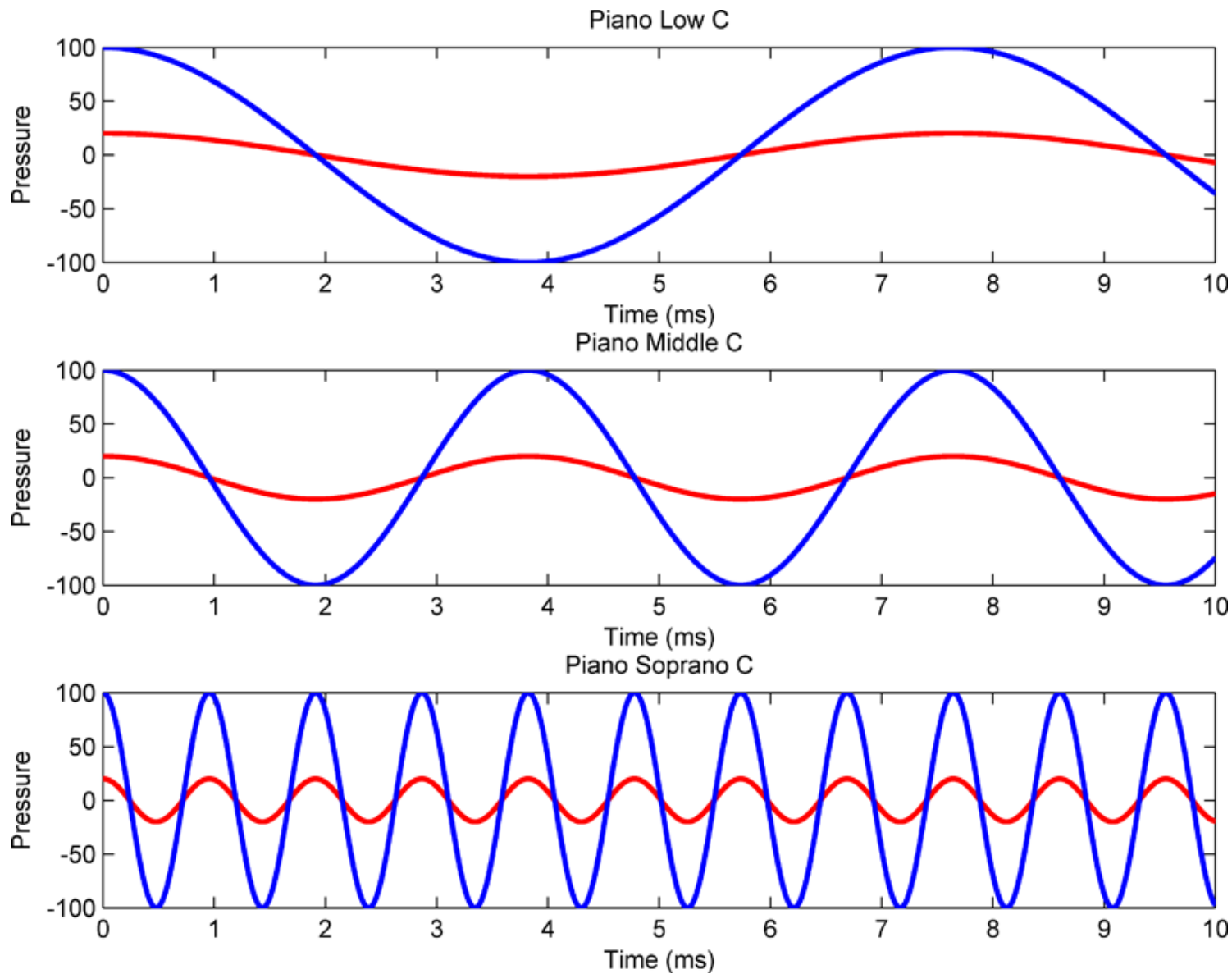
1. 스피커 작동원리
2. 능동부저
3. 수동부저

01 스피커 작동 원리

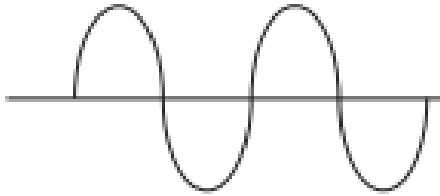
음파의 속성

- 압력파(pressure wave) : 압력 크기의 변화가 만들어 내어 매개물을 통해 전달되는 파동
 - 소리는 공기로 전달되는 압력파
 - 스피커, 드럼, 종 등 물체가 진동하면 주변의 공기도 진동
 - 공기 분자는 진동하면서 주변 분자에 에너지를 전달하고 주변의 공기 분자도 함께 진동
 - 진동 입자의 연쇄반응을 통해 인간의 고막으로 소리를 전달
- 주파수(frequency): 공기 입자가 얼마나 빨리 앞뒤로 진동하는지 나타내는 것
 - 주파수가 높아질 수록 음은 높아지고, 낮아질 수록 음도 낮아짐
 - 중간 도 음의 주파수=261.63Hz
 - 1초 동안 정확히 261.63번 진동하면서 도 음을 만든다는 뜻
- 진폭(amplitude): 진동의 크기
 - 진폭이 클 수록 소리가 크고, 진폭이 작을 수록 소리가 작음
 - 진폭은 스피커로 흐르는 전류의 양으로 조절 가능

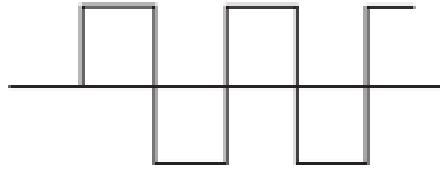
다양한 주파수와 진폭의 음파



사인파와 네모파



▶ 사인파



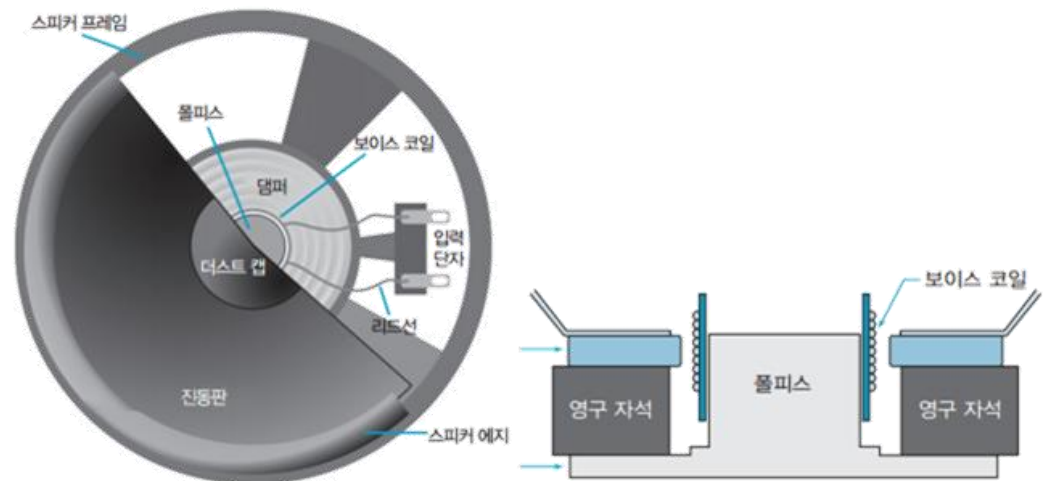
▶ 네모파

- 실제 소리는 사인 곡선 형태의 파형으로 나타냄 → 아날로그 파형
- 아두이노 우노는 실제 소리와 비슷한 사인파를 만들지 못함
- 네모파로도 소리를 만드는 압력파를 생성할 수는 있음 → 디지털 파형(네모파)
디지털 주기 파형으로 HIGH값과 LOW값 사이를 즉시 바꾸는 형태
- 아날로그 형태의 사인파처럼 소리가 예쁘지는 않다.
- MP3 등 음악이나 영화도 고해상도의 네모파로 만든 사인파
- DAC(Digital to Analog Converter)를 사용하면 고해상도의 사인파를 만들 수 있고,
컴퓨터에 저장된 모든 음악 파일은 이런 고해상도의 디지털 사인파로 구성된 것

스피커의 구조

■ 스피커의 내부 구조

- 모터와 마찬가지로 스피커도 전기를 운동으로 바꾸기 위해 전자기력을 사용
- 전압 신호(사인파 또는 네모파)가 보이스 코일로 보내지면 전류의 변화가 시작되어 자기장을 유도
- 이 자기장의 영향으로 영구자석이 보이스 코일을 끌어당겼다 밀어냈다 하면서 폴피스와 보이스코일에 연결된 진동판을 앞뒤로 떨리게 함
- 앞뒤로 움직이는 이 떨림이 스피커 앞의 공기를 진동시켜 음파가 생성되어 우리 귀까지 소리 전달



부저

■ 부저 (Buzzer, beeper)

- 단순한 소리를 내는 스피커 같은 장치
- 간단하게 소리를 생성 가능
- 아두이노 디지털 핀에 연결해서 디지털 출력으로 제어 가능

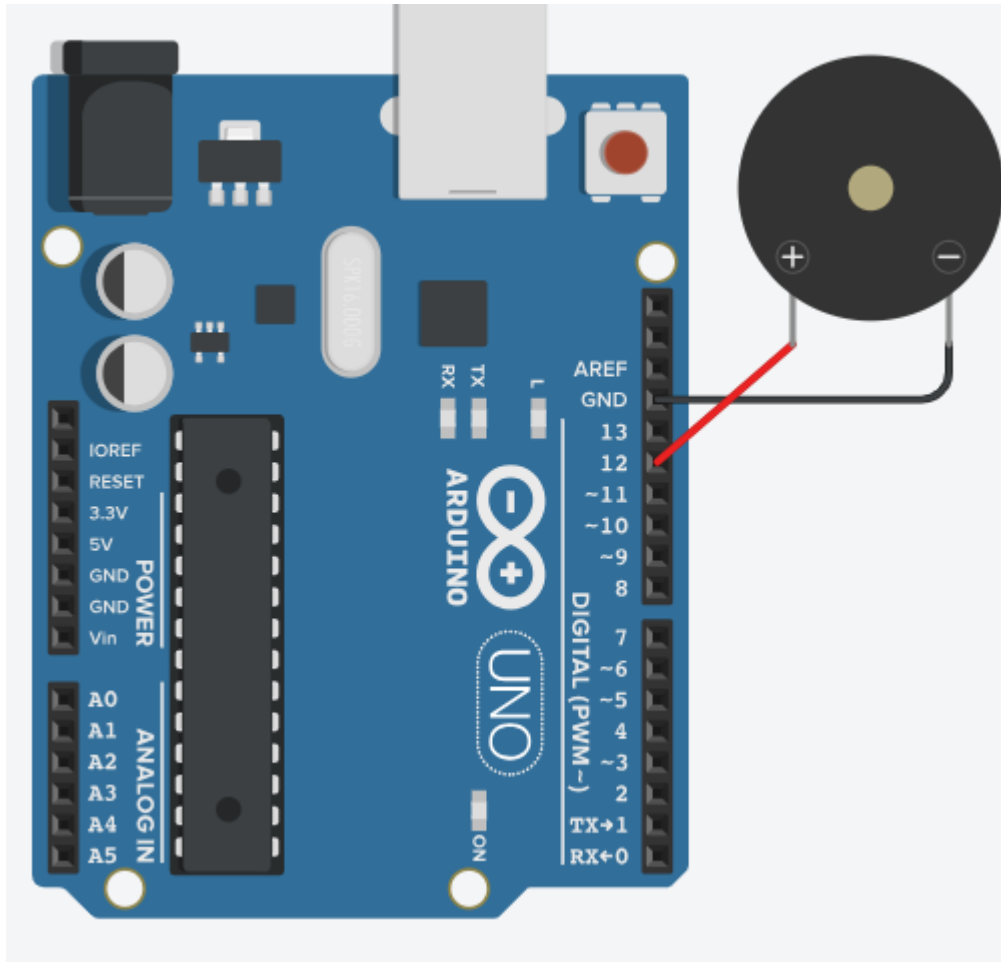
■ 능동부저 : 전류만 흐르면 자동으로 소리 출력

■ 수동부저 (피에조 부저) : 소리 + 헤르츠 지정

- 아두이노 uno의 경우 31~65,535Hz의 음 출력 가능

02 능동부저

능동부저 회로도



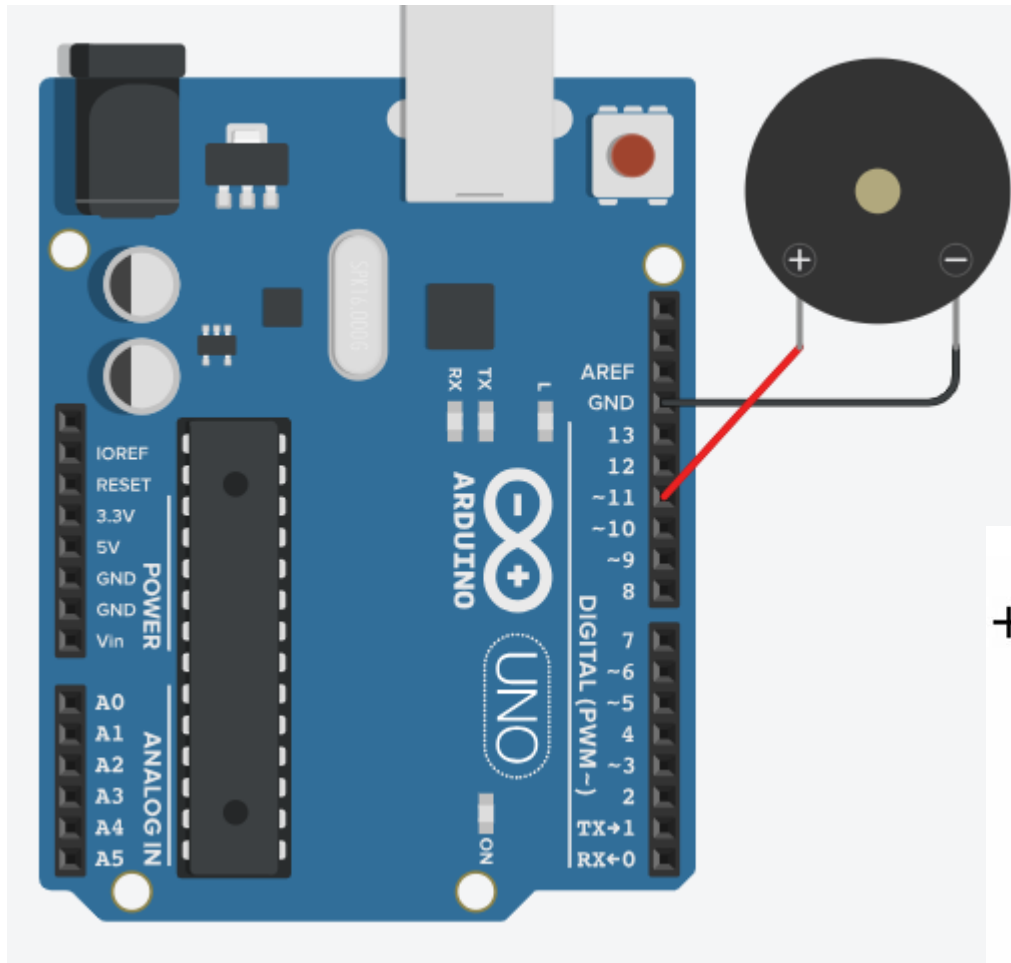
긴 쪽 +
짧은 쪽 -

uno_ex07_01: 능동부저 예제

```
1  int piezo = 12;
2
3  void setup() {
4      pinMode(piezo, OUTPUT);
5  }
6
7  void loop() {
8
9      digitalWrite(piezo, HIGH);
10     delay(1000);
11     digitalWrite(piezo, LOW);
12     delay(1000);
13
14 }
```

03 수동부저

수동부저 회로도



+/- 극성 표시되어 있음



tone() 함수

■ tone 함수

- 지정한 핀으로 지정한 주파수의 구형파(네모파) 출력
- 매개변수
 - 첫 번째 : 구형파를 출력할 핀
 - 두 번째 : 재생할 음의 주파수
 - 세 번째 : 음 재생 시간, 생략하면 noTone() 함수 호출 때까지 재생 계속
- 하드웨어 타이머를 사용하는 논블로킹(non-blocking) 함수 → “논블로킹”은 코드가 멈추지 않는다는 뜻
 - 재생을 시작하면 바로 반환하며, 다른 작업을 수행할 수 있음

tone() 함수는 이 하드웨어 타이머를 이용해

예를 들어 440Hz라면,

→ 1초에 440번 핀을 HIGH/LOW로 바꿔서 스피커를 진동함

이게 바로 소리의 진동(=주파수) 이 됨

4. tone() 함수의 한계

- tone() 함수는 PWM 신호를 사용
 - 하드웨어 타이머 사용
 - 3번과 11번 핀의 PWM 신호 출력은 사용할 수 없음(* 테스트상 되긴 함, 가급적 사용 금지)
- tone() 함수는 구형파를 사용
 - 구형파로 스피커 구동은 가능하지만, 정현파만큼 자연스럽지는 않음
 - 아두이노 듀에와 같이 DAC이 포함된 아두이노 보드 사용
 - 음악 파일 재생을 위한 확장 실드 사용
- tone() 함수는 한 번에 하나의 핀으로만 음 재생 가능
 - 여러 개 스피커를 구동하기 위해서는 타이머 관련 인터럽트를 직접 제어해야 할 수 있음

uno_ex07_02: 수동부저 예제

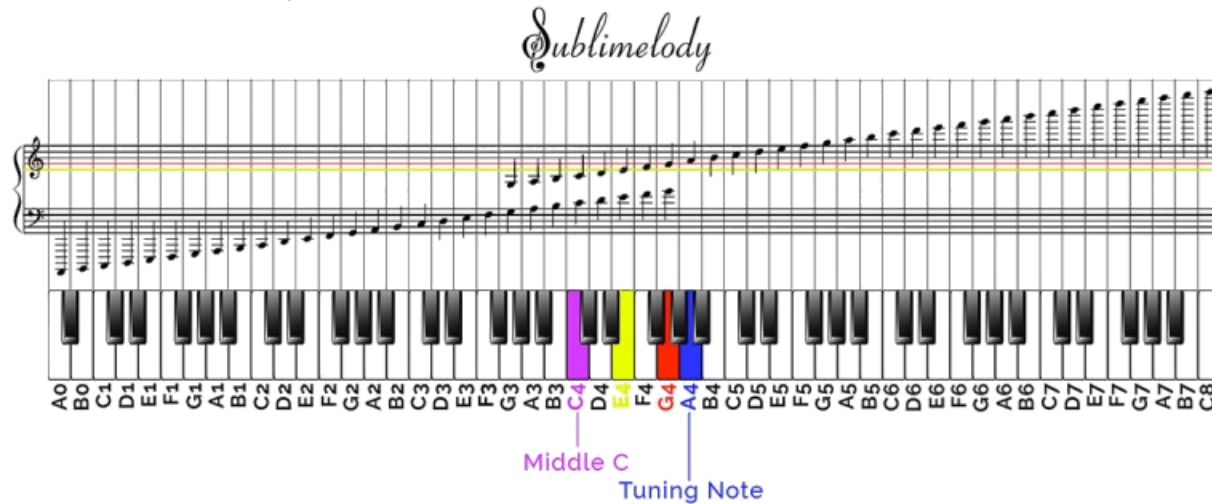
```
1  int piezo = 10;
2
3  int numTones = 8;
4  int tones[] = {261, 294, 330, 349, 392, 440, 494, 523}; // 각 주파수(Hz): 도(261), 레(294), 미(330),
5  //도 레 미 파 솔 라 시 도                                파(349), 솔(392), 라(440), 시(494), 도(523)
6
7  void setup() {
8      pinMode(piezo, OUTPUT);
9  }
10
11 void loop() {
12     delay(1000);
13
14     for (int i = 0; i < numTones; i++) {
15         tone(piezo, tones[i]);
16         delay(500);
17     }
18     noTone(piezo);
19 }
```

“pitches” 라이브러리 추가

■ pitches.h : 음계를 저장한 헤더파일

```
/*  
 * Public Constants  
 * pitches.h  
 */
```

```
#define NOTE_B0 31  
#define NOTE_C1 33  
#define NOTE_CS1 35  
#define NOTE_D1 37  
#define NOTE_DS1 39  
#define NOTE_E1 41  
.  
.  
.  
#define NOTE_CS8 4435  
#define NOTE_D8 4699  
#define NOTE_DS8 4978
```



■ “Pitches” 폴더 생성 후 하위에 “pitches.h” 파일 추가

› 내 PC › 로컬 디스크 (C:) › arduino_workspace › libraries › Pitches

이름	수정한 날짜	유형
 pitches.h	2023-08-22 오후 4:15	C/C++ Header

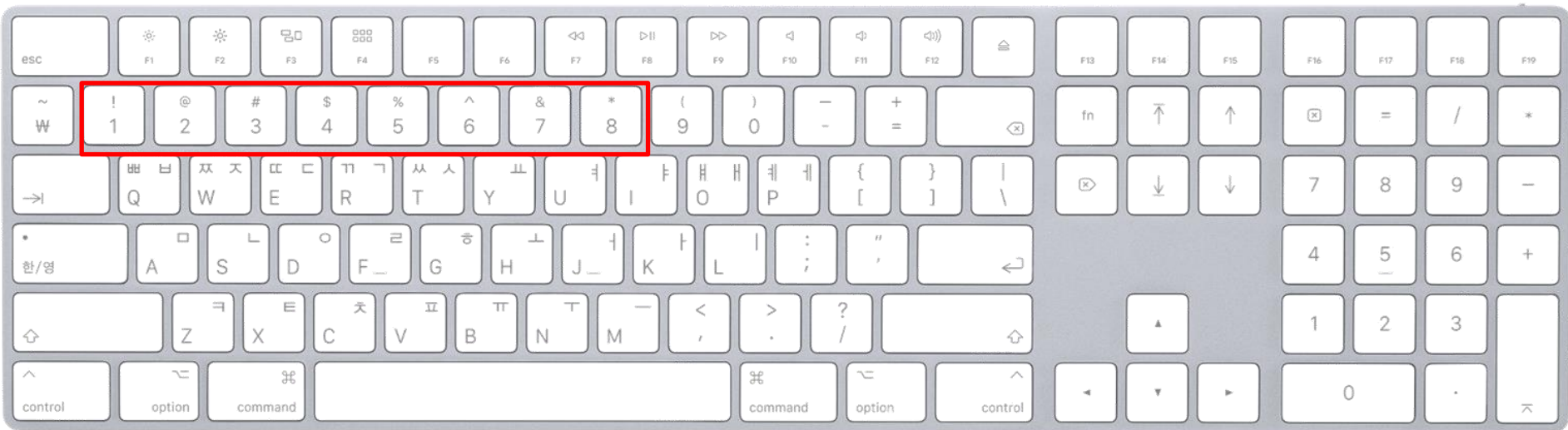
uno_ex07_03: 음악재생

```
1  #include <pitches.h> //게이름이 정의된 헤더 파일 삽입
2
3  const int SPEAKER=10; //10번 핀을 사용하는 SPEAKER 상수 정의
4
5  //연주할 음이 저장된 배열 notes 선언
6  int notes[]= {
7      NOTE_A4, NOTE_E3, NOTE_A4, 0,          0은 소리를 내지 않는 것, 즉 쉼표를 의미
8      NOTE_A4, NOTE_E3, NOTE_A4, 0,
9      NOTE_E4, NOTE_D4, NOTE_C4, NOTE_B4, NOTE_A4, NOTE_B4, NOTE_C4, NOTE_D4,
10     NOTE_E4, NOTE_E3, NOTE_A4, 0
11 };
12
13 //음의 지속 시간이 저장된 배열 times 선언. 단위는 ms
14 int times[]= {
15     250, 250, 250, 250,
16     250, 250, 250, 250,
17     125, 125, 125, 125, 125, 125, 125, 125,
18     250, 250, 250, 250
19 };
```

uno_ex07_03: 음악재생

```
21 void setup()
22 {
23   pinMode(SPEAKER, OUTPUT);
24 }
25
26 void loop()
27 {
28   //게이름, 지속 시간 배열 순서대로 음악 재생
29   for(int i=0; i<20; i++)
30   {
31     tone(SPEAKER, notes[i], times[i]);
32     delay(times[i]);
33   }
34 }
```

uno_ex07_04: 키보드 피아노



키보드	음계	아스키코드
1	도	49
2	레	50
3	미	51
4	파	52
5	솔	53
6	라	54
7	시	55
8	도	56

uno_ex07_04: 키보드 피아노

```
1  #include < pitches.h> //게이름이 정의된 헤더 파일 삽입
2
3  const int SPEAKER=10; //10번 핀을 사용하는 SPEAKER 상수 정의
4
5  int tones[] = {261, 294, 330, 349, 392, 440, 494, 523};
6  //도, 레, 미, 파, 솔, 라, 시, 도
7
8  int input = -1;
9  void setup()
10 {
11     Serial.begin(9600);
12     pinMode(SPEAKER, OUTPUT);
13 }
14
15 void loop()
16 {
17     if(Serial.available()>0){ // 시리얼로 입력이 들어오면
18
19         input = Serial.read(); // 입력(아스키 코드값) 읽기
20         input -=48; //아스키코드->숫자 변환 // '0' 문자(48)를 빼서 숫자로 변환
21
22         if(1<=input && input<=8){ // 입력값이 1~8 사이면
23
24             tone(SPEAKER, tones[input-1], 300); // 해당 음 재생
25             delay(300); // 0.3초 유지
26
27         }else if(input != -38) Serial.println("wrong input");
28     }
29 }
30
31 }
```

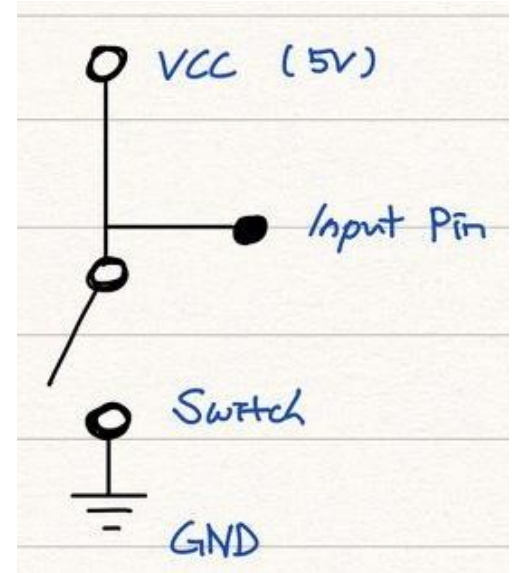
목차

1. 디지털 입력
2. 아날로그 입력
3. 다중 연결

01 디지털입력 : 택트 스위치, 가변저항

디지털 입력값을 읽는 방법

문제: 버튼을 누르면
필요한 작업을 수행하도록 하려고 한다.
어떻게 하면 버튼 눌림을
디지털 입력값으로 감지할 수 있을까?



■ 해결 방법

- 버튼을 누르면 회로가 연결되어 당연히 HIGH값으로 인식 될 것이다.
- 버튼을 누르지 않았을 때는 입력으로 지정된 해당 입출력 핀이 어디에도 연결되지 않은 채 붕 떠 있는(플로팅) 상태가 된다.
 - 핀자체의 칩에서는 0V, 5V인지 1V인지 인식할 수가 없어서 입력 단자 주위에 정전기나 잡음에 의해서 오류가 생기게 됨.
- 이 문제는 풀다운 저항으로 해결할 수 있다(풀다운 저항: 낮은 전압에 (Low) 연결되어 있는 저항)

디지털 입력값을 읽는 방법 - 풀다운 저항 사용하기

❖ Pull up 저항

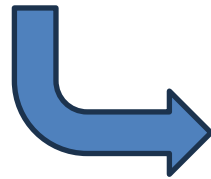
- 높은 전압 (High) 에 연결되어 있는 저항

❖ Pull down 저항

- 낮은 전압 (Low) 에 연결되어 있는 저항

❖ 플로팅 현상 (Floating)

- 연결되어 있지 않은 상태
- 전류가 흐르는지 알 수 없는 상태

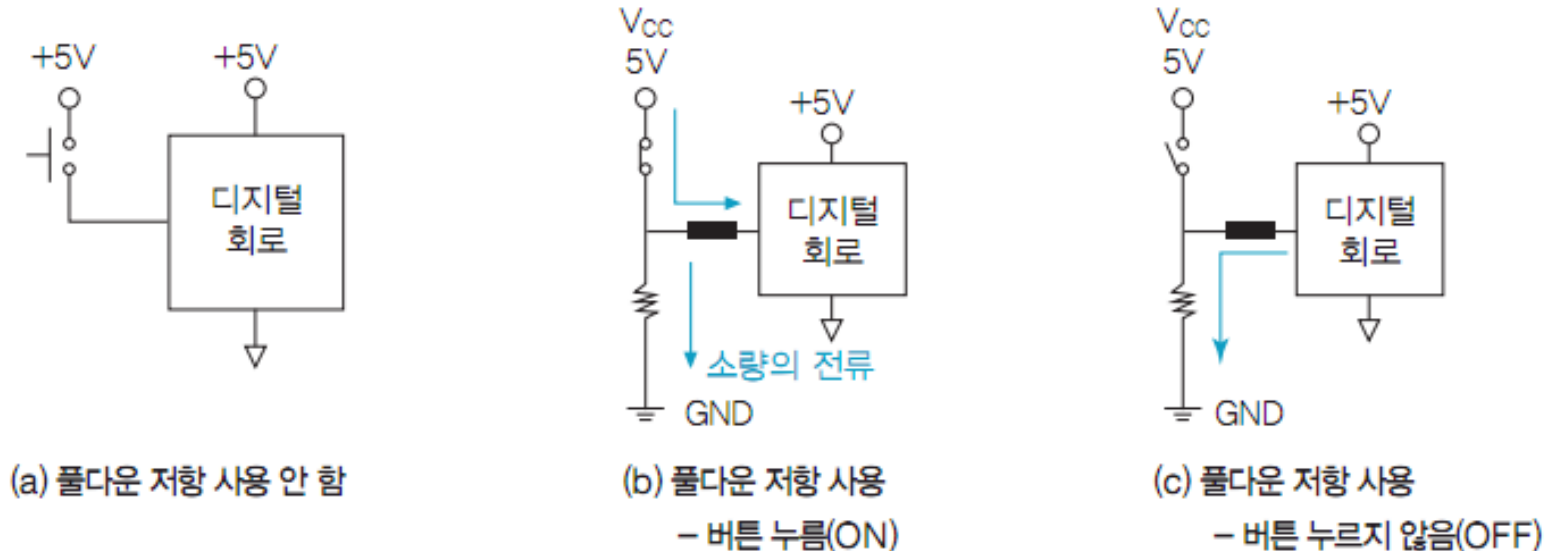


노이즈, 정전기에 취약.
잡음에 의한 오동작 가능성이 큼

안정적인 회로가 필요 ➔ 풀업 및 풀다운 저항 사용

디지털 입력값을 읽는 방법 - 풀다운 저항 사용하기

그림 2-5 버튼의 입력값과 풀다운 저항



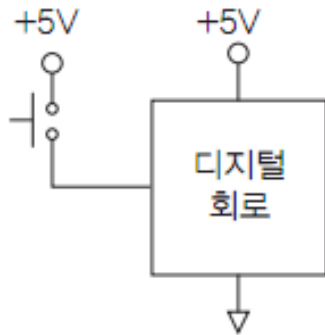
버튼 또는 스위치	누름(ON)	누르지 않음(OFF)
풀다운 저항 사용 안 함	5V(High)	플로팅
풀다운 저항 사용	5V(High)	0V(Low)

■ 풀다운 저항을 사용하면

- 버튼을 누르면 전류는 저항이 거의 없는 5V 경로를 따라 흐른다.
- 버튼을 누르지 않으면 전류는 저항이 높은 GND 경로를 따라 흐른다.
- 따라서 풀다운 저항은 단지 “전압을 0V로 잡아주는 역할”을 함

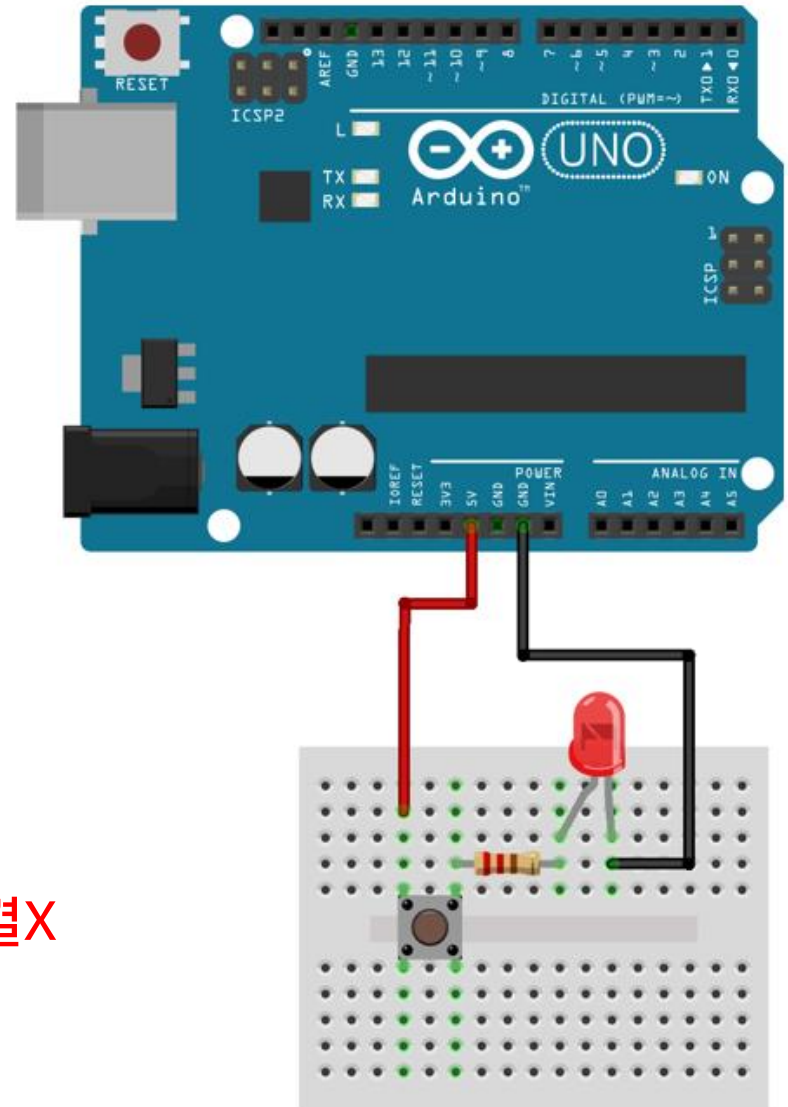
디지털 입력 회로도

- 풀다운 저항이 적용되지 않은 버튼 회로



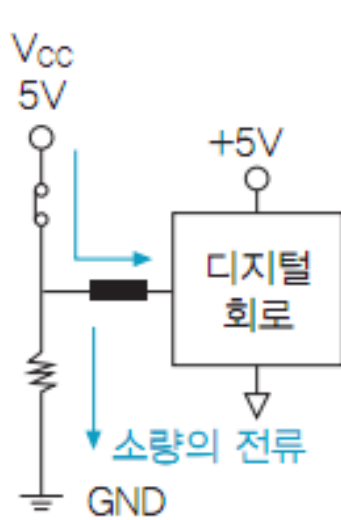
(a) 풀다운 저항 사용 안 함

- 아두이노 5V와 GND는 절대 저항없이 연결X

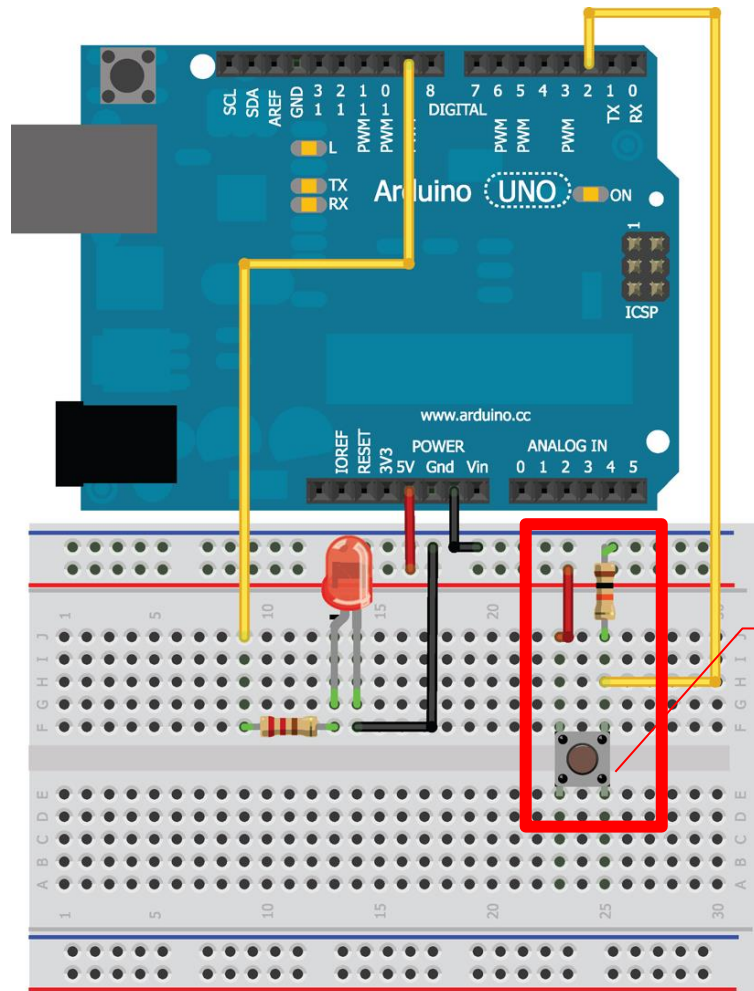


디지털 입력값을 읽는 방법 - 소프트웨어적으로 버튼-LED 제어

풀다운 저항이 적용된 버튼 회로



(b) 풀다운 저항 사용
- 버튼 누름(ON)



4개의 핀 모두
간섭없이 연결

디지털 입력값을 읽는 방법 - 소프트웨어적으로 버튼-LED 제어

```
1  const int LED=9; //9번 핀을 사용하는 LED 상수 정의
2  const int BUTTON=2; //버튼을 2번 핀에 지정
3
4  void setup()
5  {
6      pinMode(LED, OUTPUT);
7      //LED(9번 핀)을 출력으로 지정
8      pinMode(BUTTON, INPUT);
9      //BUTTON(2번 핀)을 입력으로 지정
10 }
11
12 void loop()
13 {
14     if(digitalRead(BUTTON)==LOW)
15     {
16         digitalWrite(LED, LOW);
17     }
18     else
19     {
20         digitalWrite(LED, HIGH);
21     }
22 }
```

■ If문으로 버튼 값을 검사하여 LED를 작동시킨다.

▪ 버튼이 안 눌리면

(digitalRead(BUTTON)==LOW)

LED를 끄고,

digitalWrite(LED, LOW);

▪ 그렇지 않으면(버튼이 눌리면)

LED를 켜다.

digitalWrite(LED, HIGH);

디지털 입력값을 읽는 방법 - 소프트웨어적으로 버튼-LED 제어

■ 일반 버튼 동작 (모멘터리, 순간식)

- 버튼 누르면 LED 켜짐
- 버튼 떼면 LED 꺼짐

→ 버튼을 계속 누르고 있어야 LED가 켜져 있음

■ 토글 동작

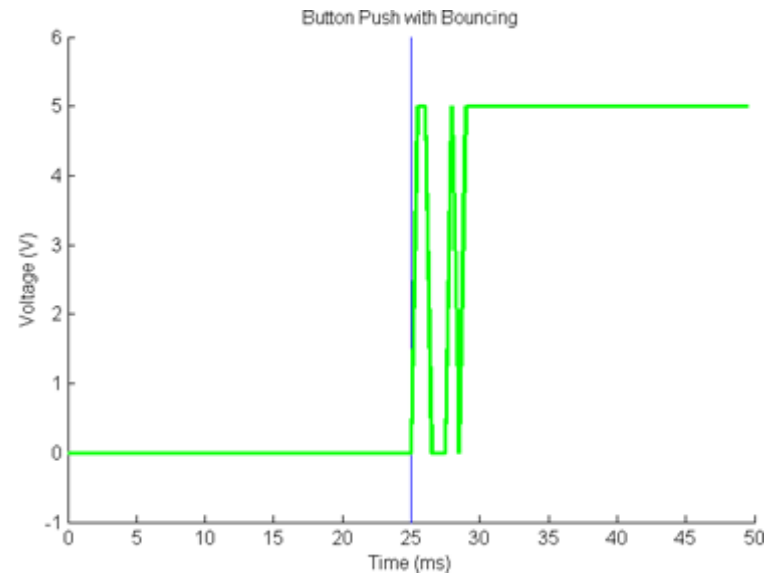
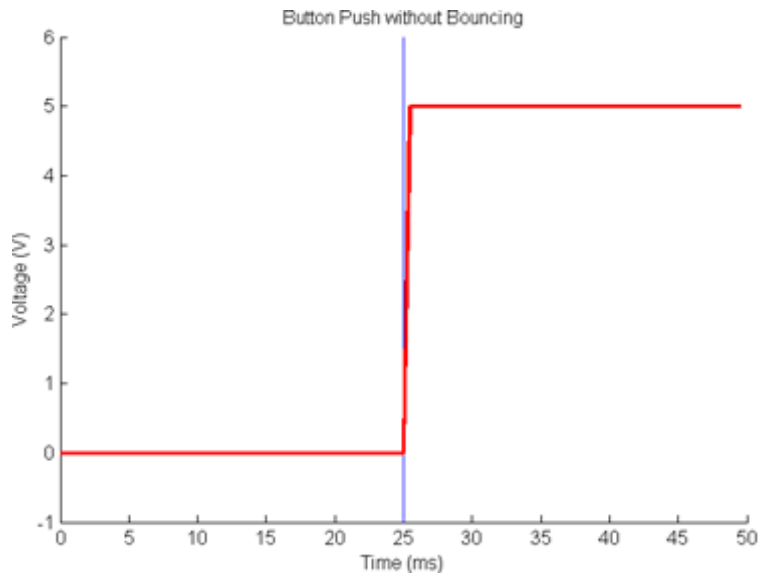
- 버튼 한 번 누르면 LED 계속 켜짐 (버튼 떼도 유지)
- 다시 버튼 누르면 LED 꺼짐
- 또 누르면 다시 켜짐 ...

→ 버튼을 “전원 스위치”처럼 쓰는 방식

스위치 바운싱(채터링 또는 진동 잡음)

■ 스위치 바운싱(채터링 또는 진동 잡음)

- 스위치 바운싱은 한 번 누른 동작이 여러 번 신호로 인식되는 현상
- 버튼의 기계적인 현상
- 값이 변경되는 것을 알아내기 어렵다.
- 실제로 버튼을 눌러도 예상대로 작동하지 않는 경우가 많다.



소프트웨어적 해결방법

■ 소프트웨어로 스위치 바운싱 처리

- 버튼의 스위치 바운싱 해결 방법
- 디바운싱이라고도 부른다.

1. 이전 버튼 상태와 현재 버튼 상태를 저장한다.
2. 현재 버튼 상태를 읽어 온다.
3. 입력값이 이전 버튼 상태와 다를 때는 바운싱이 끝날 때 까지 5ms 동안 기다린다.
4. 5ms 후에 버튼 상태를 다시 읽어서 그 값을 현재 버튼 상태로 저장한다.
5. 이전 버튼 상태가 LOW이고 현재 버튼 상태가 HIGH라면 LED 상태를 전환한다.
6. 이전 버튼 상태의 변수값에 현재 버튼 상태를 입력한다.
7. 단계 2를 반복한다.

디지털 입력값을 읽는 방법 - 스위치 바운싱 처리

```
1  const int LED=9; //9번 핀을 사용하는 LED 상수 정의
2  const int BUTTON=2; //버튼을 2번 핀에 지정
3
4  boolean lastButton=LOW; //이전 버튼의 눌림 상태를 Boolean형 변수로 선언
5  boolean currentButton=LOW; //현재 버튼의 눌림 상태를 Boolean형 변수로 선언
6  boolean ledOn=false; //현재 LED의 점멸 상태를 Boolean형 변수로 선언
7
8  void setup()
9  {
10     pinMode(LED, OUTPUT); //LED(9번 핀)을 출력으로 지정
11     pinMode(BUTTON, INPUT); //BUTTON(2번 핀)을 입력으로 지정
12 }
13
14 /*
15  * 스위치 디바운싱 함수
16  * debounce() 함수에 이전 버튼 상태를 인수로 사용하여
17  * 디바운싱 처리된 현재 버튼 상태값을 반환
18  */
19
20 boolean debounce(boolean last)
21 {
22     boolean current=digitalRead(BUTTON); //현재 버튼 상태를 확인
23     if(last!=current) //이전 버튼 상태와 현재 버튼 상태가 다름 //버튼을 누른 상태임
24     {
25         delay(5); //5ms 동안 기다림
26         current=digitalRead(BUTTON); //current에 현재 버튼 상태를 저장
27     }
28     return current; //current 변수값을 반환
29 }
```

디지털 입력 값을 읽는 방법 - 스위치 바운싱 처리

```
32 void loop()  
33 {  
34     currentButton=debounce(lastButton); //디바운싱 처리된 버튼값을 읽음  
35     if(lastButton==LOW&&currentButton==HIGH) //버튼을 누름  
36     {  
37         ledOn=!ledOn; //LED 점멸 상태를 나타내는 ledOn 변수값을 바꿈  
38     }  
39     lastButton=currentButton; //이전 버튼값을 현재 버튼값으로 변경  
40  
41     digitalWrite(LED, ledOn); //LED 점멸 상태를 바꿈  
42 }
```

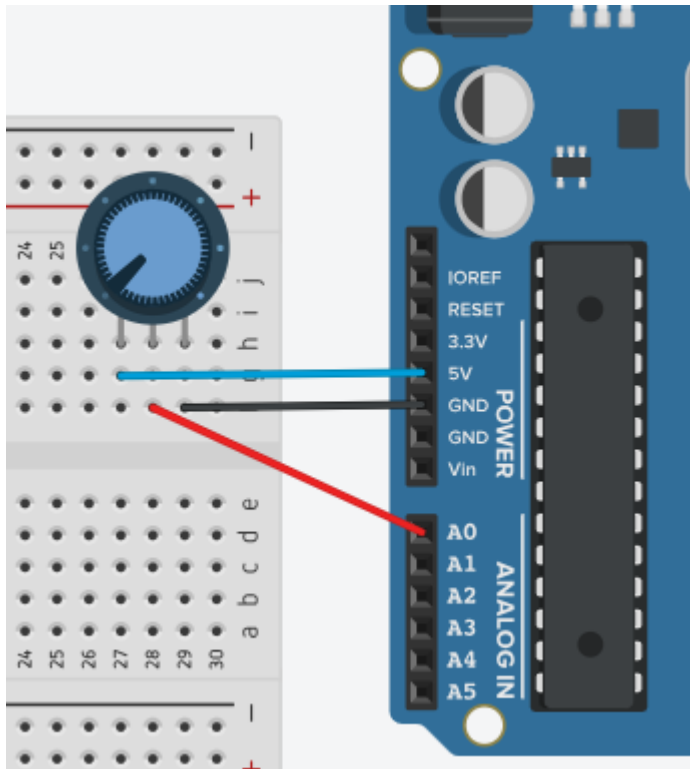
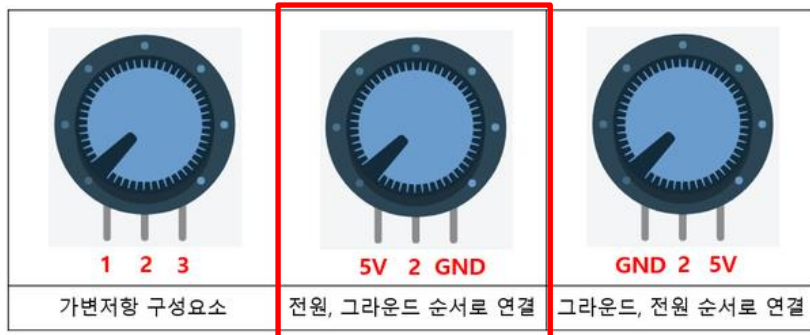
02 아날로그 입력

가변저항 사용

- 가변저항? 전류의 세기를 사용자 입력으로 변하는 저항
ex) 오디오의 볼륨을 제어하거나 LED 조명의 밝기를 제어
- 양 끝의 두 단자: 전원(5V)과 그라운드(GND)에 연결
- 가운데 단자: 와이퍼(Wiper)라고하여 가변저항의 변경된 저항 값을 읽을 수 있는 단자
- 저항은 극성이 없는 전자소자이기 때문에 전원과 그라운드는 서로 바뀌서 연결해도 무방



uno_ex6_3: 아날로그 값 읽기



```
1  const int INPUT_PIN = A0;
2
3  int input_value;
4
5  void setup() {
6      pinMode(INPUT_PIN, INPUT);
7      Serial.begin(9600);
8  }
9
10 void loop() {
11
12     input_value = analogRead(INPUT_PIN);//0~1023
13
14     Serial.println("INPUT="+String(input_value));
15     delay(500);
16
17 }
```

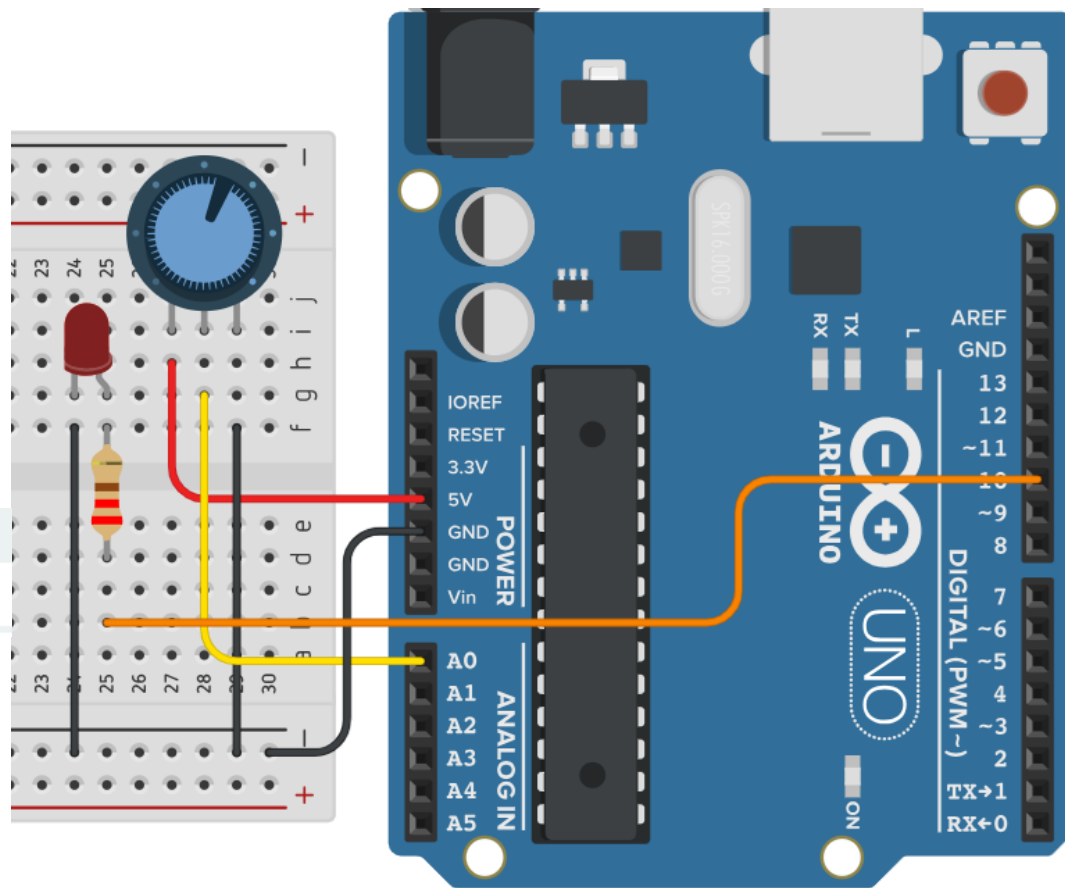
uno_ex6_4: LED 밝기 조절

모듈명	모듈 PIN	아두이노 PIN
가변저항	VCC(1)	5V
	GND(3)	GND
	INPUT(2)	A0
LED	양극	10
	음극	GND

출력 시리얼 모니터 x

Message (Enter to send message to 'Arduino Uno' on 'COM3')

INPUT=1003	OUTPUT=250
INPUT=1003	OUTPUT=250
INPUT=143	OUTPUT=35
INPUT=143	OUTPUT=35
INPUT=246	OUTPUT=61
INPUT=348	OUTPUT=87
INPUT=430	OUTPUT=107



uno_ex6_4: LED 밝기 조절

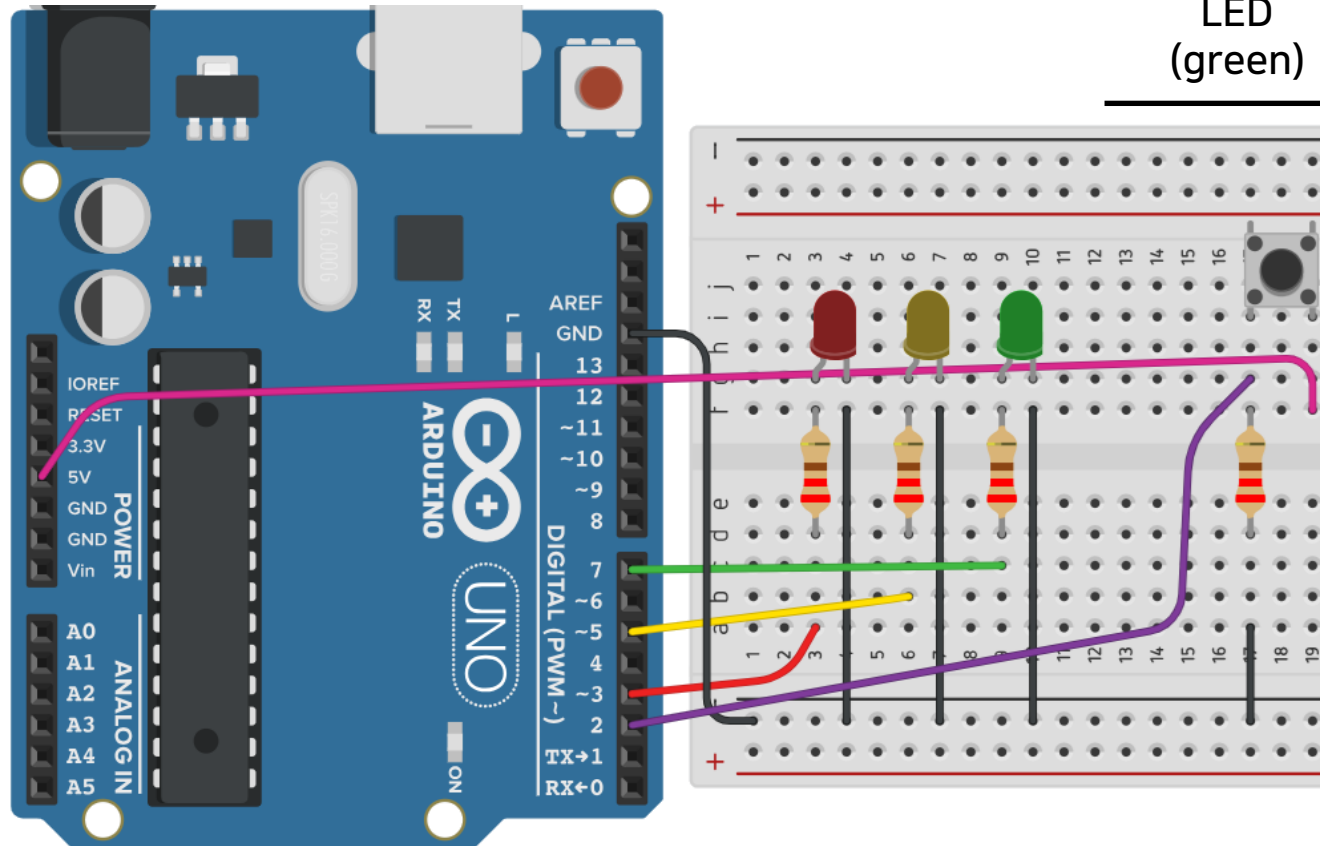
uno6_4.ino

```
1  const int INPUT_PIN = A0;
2  const int OUTPUT_PIN=10;
3  int input_value;
4  int output_value;
5
6
7  void setup() {
8      pinMode(INPUT_PIN, INPUT); //0~1023
9      pinMode(OUTPUT_PIN, OUTPUT); //0~255
10     Serial.begin(9600);
11 }
12
13 void loop() {
14
15     input_value = analogRead(INPUT_PIN);    입력 : 0 ~ 1023 범위
16     output_value = input_value/4;           출력 : 0 ~ 255 값
17
18     Serial.println("INPUT="+String(input_value)+"\t OUTPUT="+String(output_value));
19     analogWrite(OUTPUT_PIN, output_value);
20
21     delay(500);
22 }
```

03 다중 연결

입력모듈 1개 : 출력모듈 N

모듈명	모듈 PIN	아두이노 PIN
스위치	VCC	5V
	GND	GND
	INPUT	2
LED (red)	양극	3
	음극	GND
LED (yellow)	양극	5
	음극	GND
LED (green)	양극	7
	음극	GND



입력모듈 1개 : 출력모듈 N

uno6_5.ino

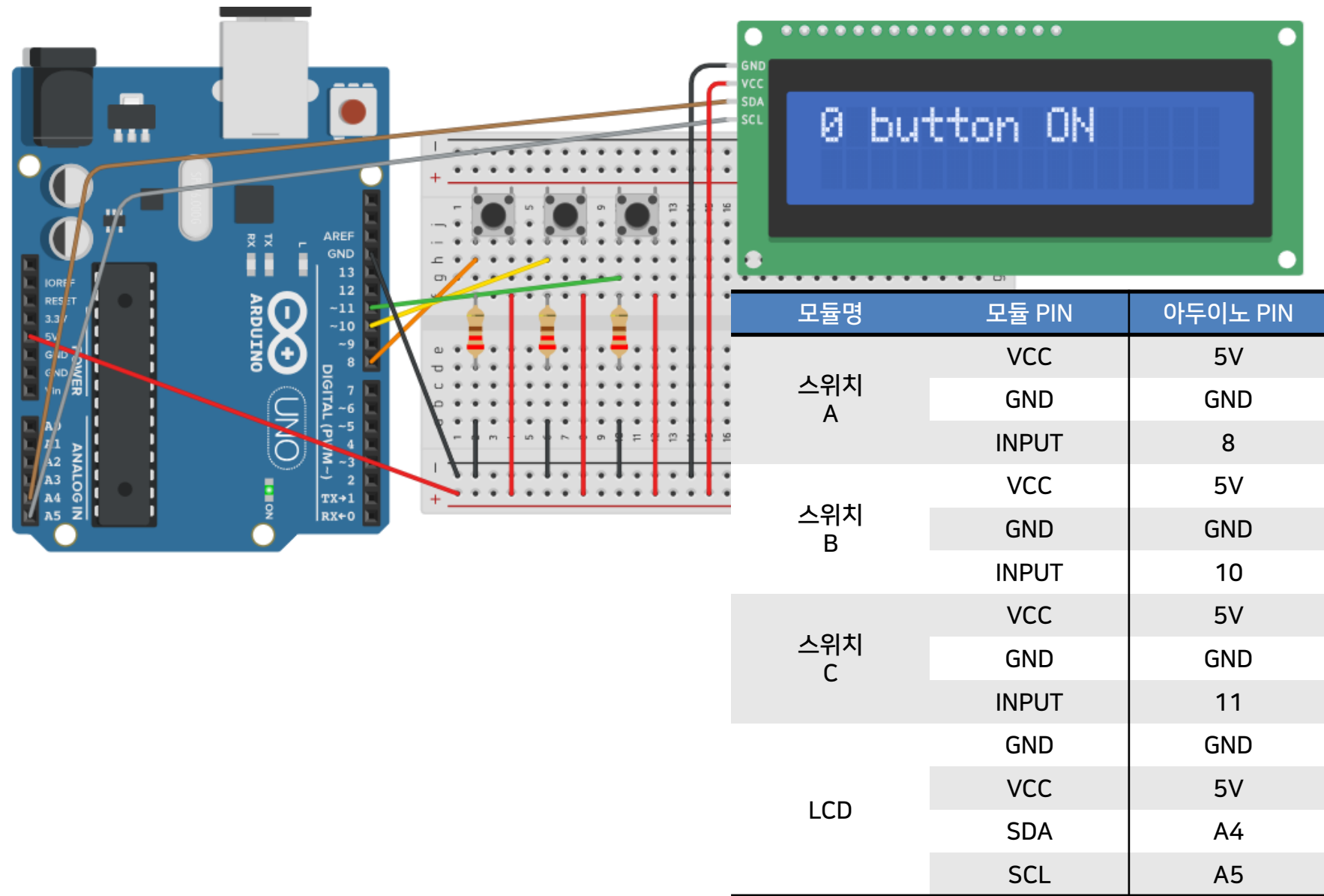
```
1  const int INPUT_PIN=2; //버튼을 2번 핀에 지정
2  const int OUTPUT_PIN[]={3,5,7}; //3,5,7번 핀을 사용하는 LED 상수 정의
3
4  boolean lastButton=LOW; //이전 버튼의 눌림 상태를 Boolean형 변수로 선언
5  boolean currentButton=LOW; //현재 버튼의 눌림 상태를 Boolean형 변수로 선언
6  int count=0;
7  void setup()
8  {
9      pinMode(INPUT_PIN, INPUT);
10     for(int i=0; i<3; i++)pinMode(OUTPUT_PIN[i], OUTPUT);
11     Serial.begin(9600);
12 }
13
14 boolean debounce(boolean last)
15 {
16     boolean current=digitalRead(INPUT_PIN); //현재 버튼 상태를 확인
17     if(last!=current) //이전 버튼 상태와 현재 버튼 상태가 다름
18     {
19         delay(5); //5ms 동안 기다림
20         current=digitalRead(INPUT_PIN); //current에 현재 버튼 상태를 저장
21     }
22     return current; //current 변수값을 반환
23 }
```

입력모듈 1개 : 출력모듈 N

```
25 void loop()
26 {
27     currentButton=debounce(lastButton); //디바운싱 처리된 버튼값을 읽음
28     if(lastButton==LOW&&currentButton==HIGH) //버튼을 누름
29     { //순간적으로 버튼이 안 눌리다가(LOW → HIGH 변화가 생길 때)" 버튼을 눌렀다고 인식
30         for(int i=0; i<3; i++)digitalWrite(OUTPUT_PIN[i], 0); //모든 LED 끄기
31         if(count==2)count=0;
32         else count++;
33         Serial.println(count);
34     }
35     lastButton=currentButton; //이전 버튼값을 현재 버튼값으로 변경
36         //lastButton을 현재 상태로 변경
37     digitalWrite(OUTPUT_PIN[count], 1); //LED 점멸 상태를 바꿈
38 }
39
```

버튼을 누를 때마다 LED가 0 → 1 → 2 → 다시 0 순으로 켜짐.
Serial.println(count);로 현재 LED 인덱스가 시리얼 모니터에 출력됨.

입력모듈 N개 : 출력모듈 1개



입력모듈 N개 : 출력모듈 1개

```
1 #include <LiquidCrystal_I2C.h>
2
3 //initialize lcd
4 LiquidCrystal_I2C lcd(0x20,16,2);
5
6 //스위치
7 const int BTN[]={8,10,11};
8
9 boolean lastButton[]={0,0,0}; //이전 버튼의 눌림 상태를 Boolean형 변수로 선언
10 boolean currentButton[]={0,0,0}; //현재 버튼의 눌림 상태를 Boolean형 변수로 선언
11 int choice=0;
12
13 void debounce()
14 {
15
16     for(int i=0; i<3; i++) {
17         currentButton[i] = digitalRead(BTN[i]);
18
19         if(lastButton[i]!=currentButton[i]) //이전 버튼 상태와 현재 버튼 상태가 다름
20         {
21             delay(5); //5ms 동안 기다림
22             currentButton[i] = digitalRead(BTN[i]);
23         }
24     }
25 }
26
```

0X27로 변경

입력모듈 N개 : 출력모듈 1개

```
27 void setup()
28 {
29     //lcd.begin(16,2);
30     lcd.init();
31     lcd.clear();
32     lcd.backlight();
33
34     //버튼 초기화
35     for(int i=0; i<3; i++)
36     |   pinMode(BTN[i], INPUT);
37
38 }
39
40 void loop()
41 {
42     debounce(); //디바운싱 처리된 버튼값을 읽음
43     for(int i=0; i<3; i++){
44         if(lastButton[i]==LOW&&currentButton[i]==HIGH) //버튼을 누름
45         {
46             lcd.clear();
47             lcd.print((String)i+" button ON");
48
49             delay(300);
50         }
51         lastButton[i]=currentButton[i]; //이전 버튼값을 현재 버튼값으로 변화
52     }
53 }
```

요약

- 디지털 입력
- 아날로그 입력
- 다중 연결 (입력모듈:출력모듈)
 - 1:N
 - N:1

Thank You